Part 4: Understanding Data Sets

Summary

This Part presents an example of reading in a data set, and ensuring the data is fully understood and properly represented prior to any further analysis.

The ability to read in, clean, and understand data sets is a crucial skill. Data do not typically come in a convenient form. Careful consideration should be given to exploring the data to ensure full understanding of the nature of the data set.

Example Data Set

First, we will read in a data set that we will use in our following examples.

Visit the website

https://www.sec.gov/opa/data/market-structure/marketstructuredownloadshtml-by_security.html

and download the data from the first quarter of 2017. Create a directory for working on these examples, and place this file in that directory.

These data come from the U.S. Securities and Exchange Commission repository on market structure. This data set contains daily characteristics of a range of market variables over 5000 equities and ETFs, accumulated over several exchanges.

Be sure that the working directory is correctly set in R.

Now use the `read.table()` function to read in the file:

```
> fulldata = read.table("q1_2017_all.csv", sep=",",
+                       header=T,quote="")
```

This is a "csv" file, i.e., there are "comma-separated-values". This leads to the use of `sep=","`. The `header=T` argument specifies that the first line of the file gives the variable names.

Exercise: What happens if you use `read.table()` in the example above without `quote=""`?

Understanding the Variables

Whenever a new data set is encountered, one should fully understand its variables. Specific questions to address for each variable:

1. Describe the information that is stored in this column.
2. What type of variable is it? (Timestamp, ordinal factor, categorical factor, ratio scale, other?) Be sure that R represents the variable correctly.
3. Are there any missing values? What is the source of the missingness?
4. Are there any extreme/inappropriate values? What is the source of the extreme value?

Look at the variables:

```
> names(fulldata)
 [1] "Date"                      "Security"
 [3] "Ticker"                    "McapRank"
 [5] "TurnRank"                  "VolatilityRank"
 [7] "PriceRank"                 "LitVol..000."
 [9] "OrderVol..000."            "Hidden"
[11] "TradesForHidden"           "HiddenVol..000."
[13] "TradeVolForHidden..000."   "Cancels"
[15] "LitTrades"                 "OddLots"
[17] "TradesForOddLots"          "OddLotVol..000."
[19] "TradeVolForOddLots..000."
```

Background on the data can be found in the associated `README` file, available on Canvas in the "Data Sets" folder in the "Files" page. We will reference portions of it below.

Column 1: Date

The first column in `fulldata` is a date stamp for the observation. R has special functions for handling dates which will prove useful later when working with time series.

The `as.Date()` function transforms into the date format:

```
> print(fulldata$Date[1])   # This is Jan. 3, 2017
[1] 20170103
```

This is numeric

convert numeric into string

```
> fulldata$Date = as.Date(as.character(fulldata$Date),
+                          format="%Y%m%d")
```

Year month data

Note that it is necessary to cast the dates as strings, and then specify the format of those strings.

Exercise: Explain the use of the `format` argument to `as.Date()`.

help(strptime): gives you a list, telling you how to do the conversion

The original string was of the form 201700103

year, 4 digits, use %Y
month, 2 digits, use %m,
day, 2 digits, use %d

See help(weekdays) for some examples of working with dates

Columns 2 and 3: Security and Ticker

The variables `Security` and `Ticker` are already appropriately treated as factors by R:

```
> is.factor(fulldata$Sec) & is.factor(fulldata$Tick)
[1] TRUE
```

We also note that `Security` is either `Stock` or `ETF` (Exchange Traded Fund), and that there are over 5,000 equities under consideration:

```
> levels(fulldata$Security)
[1] "ETF"   "Stock"
```

```
> nlevels(fulldata$Ticker)
[1] 5338
```


Exercise: How many of the different ticker symbols are ETFs?

length(unique(filter(fulldata, Security == “ETF”)$Ticker))

A unique function apply to the result after filter function

We get 1642

filter is associated with time series,
so have to load dplyr package first
install.package(“dplyr”)

Exercise: Execute the commands below, and discuss what you find:

```
> length(unique(fulldata$Date))
> table(table(fulldata$Ticker))
> which.max(table(fulldata$Ticker))
> fulldata[duplicated(fulldata[,c(1,3)]),]
```

There are 62 unique dates in the data set

The table function will take the variable and return
contingency table of the counts at each combination of factor levels

The table indicates that a couple symbols appear more than 62 times. NOT GOOD

The duplicated function on col 1 and col 3 -> looks at those cols, tells me either T or F ,
did that observations already appear in the data set -> Return T or F

We determine that there are three duplicated rows, in each case, the first of the pair
has unexplained missing values, are represented as NA

We will remove the duplicated lines using the following command:

```
> fulldata = fulldata[!duplicated(fulldata[,c(1,3)],
+                                 fromLast=TRUE),]
```

by default, fromLast = FALSE, start from bottom and go up

Note that by using `fromLast=TRUE`, we remove the **first** of each pair of duplicates.

Give rows where duplicated return False
Want to keep all the unique rows, looking for their
first appearence.

Because good duplicated row is at the bottom, so I want to work my way up,
in order to mark the good rows with duplicated = F

Columns 4 through 7: Rank Variables

The next four columns provide ranks of the stocks/ETFs with respect to key attributes of Market Capitalization, Turnover, Volatility, and Price.

In general, we would prefer to start with data in its most "raw" format, and one could construct variables such as these from that data. Alas, we are often left to work with what is avaiable.

Exercise: Inspect the output of

```
> table(fulldata$VolatilityRank, fulldata$Security,
+       fulldata$Date)
```

What does this tell us about the structure of these variables? Why should we be very careful with these variables?

For each date, EFT are categorized into 4 bins, each stock is placed into 10 bins

Output:

ETF	Stock
405	362
404	362
404	362
404	362
0	362

....

Each day, the ETF's are ranked on these quantities, and divided into 4 equally-sized bins. Likewise, the stocks are divided into 10 bins

Note that a "4" for an ETF is very different than a "4" for a stock

We will change these into ordered factors to reflect their ordinal nature:

use ordered=T to specify an ordering

```
> fulldata$McapRank =
+   factor(fulldata$McapRank, ordered=TRUE)
> fulldata$TurnRank =
+   factor(fulldata$TurnRank, ordered=TRUE)
> fulldata$VolatilityRank =
+   factor(fulldata$VolatilityRank, ordered=TRUE)
> fulldata$PriceRank =
+   factor(fulldata$PriceRank, ordered=TRUE)
```


Columns 8 thru 19: The Count Variables

The remaining column in the data set consist of trade and share counts of different types.

Note that some of the volume counts are reported in 1000's of shares, hence there are non-integer values among the counts.

The `summary()` function is a useful first step in understanding the basic properties of the variables. A primary concern is the presence of extreme or impossible values.

```
> summary(fulldata[,8:19])
  LitVol..000.        OrderVol..000.          Hidden          TradesForHidden
 Min.   :     0.00   Min.   :      0   Min.   : -143.0   Min.   :     0
 1st Qu.:     3.46   1st Qu.:    885   1st Qu.:   10.0   1st Qu.:    41
 Median :    44.57   Median :   4003   Median :   75.0   Median :   501
 Mean   :   431.76   Mean   :  35747   Mean   :  454.4   Mean   :  3476
 3rd Qu.:   254.41   3rd Qu.:  15581   3rd Qu.:  370.0   3rd Qu.:  2846
 Max.   :119014.06   Max.   :7293901   Max.   :89335.0   Max.   :533874
 HiddenVol..000.    TradeVolForHidden..000.     Cancels
 Min.   :-7903.64   Min.   :     0.00        Min.   :      0
 1st Qu.:    1.15   1st Qu.:     5.64        1st Qu.:   3126
 Median :    8.74   Median :    55.31        Median :  15583
 Mean   :   66.91   Mean   :   498.04        Mean   :  73579
 3rd Qu.:   44.02   3rd Qu.:   301.58        3rd Qu.:  60112
 Max.   :14821.94   Max.   :133714.59        Max.   :7607714
```



```
   LitTrades           OddLots         TradesForOddLots OddLotVol..000.
 Min.   :     0   Min.   :     0   Min.   :     0   Min.   :   0.000
 1st Qu.:    23   1st Qu.:     8   1st Qu.:    39   1st Qu.:   0.280
 Median :   377   Median :   154   Median :   464   Median :   5.254
 Mean   :  2708   Mean   :   916   Mean   :  3115   Mean   :  32.492
 3rd Qu.:  2186   3rd Qu.:   887   3rd Qu.:  2558   3rd Qu.:  31.083
 Max.   :383222   Max.   :130106   Max.   :462414   Max.   :3728.885
 TradeVolForOddLots..000.
 Min.   :     0.00
 1st Qu.:     5.31
 Median :    49.94
 Mean   :   422.78
 3rd Qu.:   260.18
 Max.   :108034.79
```


Exercise: Do any of these variables take values that would seem to be impossible? Can you speculate as to the source of these values?

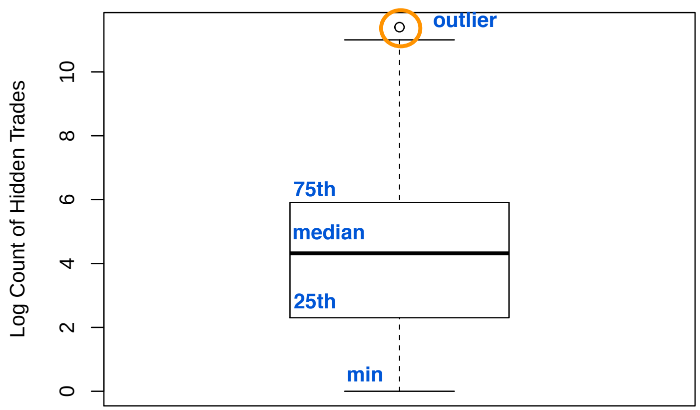
There are negative values for the "hidden" variables -> col name

Important to determine why this is happening

Of course, graphical tools are useful for understanding the properties of variables. This topic will be covered more fully in the next Part, but here we consider a classic approach: the boxplot:

```
> boxplot(log(fulldata$Hidden),
+       ylab="Log Count of Hidden Trades")
```

The "box" of the plot extends from the 25th to the 75th percentile of the data, while the solid line in the middle of the box is at the median. The two "arms" extend to the minimum and maximum value, **except** that any value labelled an outlier is plotted separately.



Exercise: Why was the logarithm of the variable utilized in this case? What issues did that create?

The question to be answered: Is there a natural explanation for the outliner?

which.max(fulldata$Hidden) -> returns 212803 fulldata[212803,]

filter(fulldata, Ticker == “SNAP”)

This IPO for Snapchat that day, —> a reasonable explanation

Apply the rank function to each of the columns,
apply(fulldata[,8:9],2,rank)[which.max(fulldata$Hidden),] pull up the row corresponds to max

shows that this observation is extreme in many variables, so the data appear to be legitimate

Exercise: Consider the outlier in the previous plot. Is this observation also extreme in the other variables? Can you find the source of this behavior?

The original dist had a long upper tail, this is quite skewed

the log transformation has the effect of “pulling this down”

When we look at y=log(x) plot, its flat out

We use log transformation a lot for that purpose

The problem is that a few 0’s and negative values. These become NA and are omitted form the boxplot.

Can use log(1+count) when 0’s are possible, this seems a little bit arbitary, but works very well

Exercise: Comment on the output of

```
> apply(fulldata[,8:19],2,which.max)
```

Missing Data

Most data sets contain some amount of missing data, for many reasons.

R uses the special symbol `NA` to represent a missing value.

As a first step, it is important to recognize how your data set represents missing values, so that R handles them properly. The function `read.table()` has an argument `na.strings` to allow one to specify missing values. **Be careful:** It is not unusual for data sets to use numbers such as `-999` to indicate a missing value.

In a `.csv` file such as that we are using here, it is common to simply have blank values to indiciate missingness, i.e., there are consecutive commas without a value between.

Exercise: Does R appropriately handle cases where a missing value is indicated using blank values in a `.csv` file?

Note that read.table() treats blanks as NA in numeric and logical columns

But for character vectors, blanks become “”(empty string), not NA

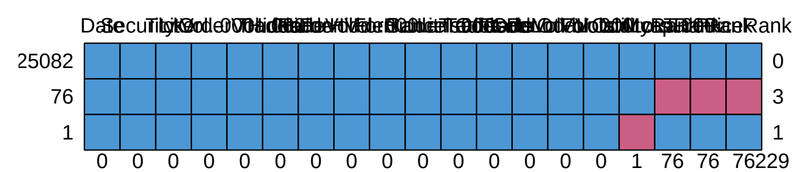
Can specify na.strings=“” to get around this

A useful tool is the function `md.pattern()` that is part of package `mice`. This function shows the different "patterns" of missingness in the data set, and how often they occur.

Consider the example on the following page, the result of calling

```
> md.pattern(data.frame(fulldata))
```

This shows that there are 325,082 complete cases in the data set. There is one case where `VolatilityRank` is missing, and there are 76 cases where `McapRank`, `TurnRank` and `PriceRank` are missing.



The blue means it is complete,
the first row corresponds to 325082 here

```
       Date Security Ticker LitVol..000. OrderVol..000. Hidden
325082    1        1      1            1              1      1
76        1        1      1            1              1      1
1         1        1      1            1              1      1
          0        0      0            0              0      0
       TradesForHidden HiddenVol..000. TradeVolForHidden..000. Cancels
325082               1               1                       1       1
76                   1               1                       1       1
1                    1               1                       1       1
                     0               0                       0       0
       LitTrades OddLots TradesForOddLots OddLotVol..000.
325082         1       1                1               1
76             1       1                1               1
1              1       1                1               1
               0       0                0               0
       TradeVolForOddLots..000. VolatilityRank McapRank TurnRank PriceRank
325082                        1              1        1        1         1
76                            1              1        0        0         0
```

Check why this is failing in these cases

```
1                              1              0        1        1         1
                               0              1       76       76        76

325082   0
76       3
1        1
       229
```


Exercise: Inspect the output from

```
> fulldata[!complete.cases(fulldata),]
```

Get the rows which contains missing values

What does this say about the cases with missing values?

We see that the " missing cases: are mostly days where these stocks had minimal activity. Maybe there is a threshhold for trading activity in order to calculate volatility.

Of course, more sophisticated visualization of the cases with missing data would likely provide more information as to the source of the missingness. In the next Part we will consider such tools.